

Go. Chapter IX: Concurrency with Shared Variables

NEDO-OFFSEC

TABLE OF CONTENT

I	Data Races	3
	1. Что такое состояние гонки (race condition).....	3
	2. Гонка данных (data race).....	3
	3. Пример: банковский счёт.....	3
	4. Как это выглядит.....	3
	5. Три способа избежать гонки.....	3
	6. Пример: ограничение через каналы.....	3
	7. Ключевая мысль.....	4
II	Mutual Exclusion (sync.Mutex)	5
	1. Проблема.....	5
	2. Решение: sync.Mutex.....	5
	3. Критический раздел (critical section).....	5
	4. defer для Unlock().....	5
	5. Мьютекс не реентерабельный.....	5
	6. Паттерн: монитор.....	5
	7. Ключевая мысль.....	5
III	Read/Write Mutex (sync.RWMutex)	6
	1. Проблема.....	6
	2. Решение sync.RWMutex.....	6
	3. Правила.....	6
	4. Когда использовать.....	6
	5. Важно.....	6
	6. Ключевая мысль.....	6
IV	Memory Synchronization	7
	1. Проблема.....	7
	2. Пример: недетерминированное поведение.....	7
	3. Почему так происходит.....	7
	4. Решение.....	7
	5. Правила.....	7
	6. Ключевая мысль.....	7
V	sync.Once	8
	1. Проблема.....	8
	2. Решение: sync.Once.....	8
	3. Как работает Do.....	8
	4. Почему лучше, чем мьютекс.....	8
	5. Итог.....	8
VI	Race Detector	9
VII	Concurrent Cache	10
VIII	Goroutines vs Threads	11

Go. Chapter IX: Concurrency with Shared Variables

В предыдущей главе мы представили несколько программ, которые используют go-подпрограммы и каналы для выражения параллелизма непосредственным и естественным путем. Однако, поступая так, мы затушевывали целый ряд важных и тонких вопросов, которые программисты должны учитывать при написании параллельного кода.

В этой главе мы более детально рассмотрим механику параллелизма. В частности, мы укажем на некоторые из проблем, связанных с совместным использованием переменных несколькими go-подпрограммами, и рассмотрим аналитические методы распознавания этих проблем и шаблоны их решений. Наконец мы поясним некоторые технические различия между go-подпрограммами и потоками операционной системы.

Data Races

Что такое состояние гонки (race condition)

Состояние гонки – ситуация, когда программа даёт неправильный результат при некотором чередовании операций нескольких горутин.

Гонка данных (data race)

Гонка данных возникает, когда две горутины одновременно обращаются к одной переменной, и хотя бы одно обращение – запись.

Пример: банковский счёт

```
var balance int

func Deposit(amount int) {
    balance = balance + amount // ЧТЕНИЕ + ЗАПИСЬ
}

func Balance() int {
    return balance
}
```

Проблема: `balance = balance + amount` – это две операции:

- Чтение `balance`
- Запись новой суммы
- Если две горутины вызовут `Deposit` одновременно, может произойти гонка.

Как это выглядит

Горутина А: читает `balance (0)` -> `balance + 200 = 200` Горутина Б: читает `balance (0)` -> `balance + 100 = 100`
Горутина А: записывает 200 Горутина Б: записывает 100 Результат: 100, хотя должно быть 300.

Три способа избежать гонки

- Не записывать переменную – только чтение.
- Ограничить переменную одной горутиной (например, через каналы).
- Взаимное исключение – допускать доступ только одной горутине за раз (Mutex).

Пример: ограничение через каналы

```
var deposits = make(chan int)
var balances = make(chan int)

func Deposit(amount int) { deposits <- amount }
func Balance() int      { return <-balances }

func teller() {
    var balance int
    for {
```

```
        select {
        case amount := <-deposits:
            balance += amount
        case balances <- balance:
        }
    }
}

func init() {
    go teller()
}
```

Ключевая мысль

Гонка данных – это всегда ошибка. Нет “доброкачественных” гонок. Избегай их тремя способами: не пиши, ограничь горутины, или используй мьютексы.

Mutual Exclusion (sync.Mutex)

Проблема

Гонка данных возникает, когда несколько горутин одновременно обращаются к одной переменной, и хотя бы одна из них пишет.

Решение: sync.Mutex

Мьютекс обеспечивает взаимное исключение – доступ к критическому разделу получает только одна горутина за раз.

```
import "sync"

var mu sync.Mutex
var balance int

func Deposit(amount int) {
    mu.Lock()           // захват блокировки
    balance += amount
    mu.Unlock()         // освобождение блокировки
}

func Balance() int {
    mu.Lock()
    defer mu.Unlock()  // освобождение через defer
    return balance
}
```

Критический раздел (critical section)

Область между Lock() и Unlock() – это критический раздел. В нём горутина может безопасно читать и изменять защищаемые переменные.

defer для Unlock()

Используй defer mu.Unlock() сразу после Lock() – это гарантирует освобождение блокировки даже при панике или раннем возврате.

Мьютекс не реентерабельный

Мьютекс нельзя заблокировать повторно в той же горутине. Это приведёт к взаимоблокировке (deadlock).

```
func Withdraw(amount int) bool {
    mu.Lock()
    defer mu.Unlock()
    Deposit(-amount) // deadlock! Deposit пытается захватить тот же мьютекс
    // ...
}
```

Решение: разделить логику на внутреннюю функцию (без блокировки) и внешнюю (с блокировкой).

Паттерн: монитор

Набор экспортируемых функций, инкапсулирующих переменные и выполняющих блокировку/освобождение.

Ключевая мысль

Мьютекс – это простой и надёжный способ защитить общие данные. Главное правило: всегда освобождай блокировку через defer.

Read/Write Mutex (sync.RWMutex)

Проблема

Когда много горутин читают данные, а пишут редко, обычный `sync.Mutex` блокирует всех, даже читателей, хотя они не мешают друг другу.

Решение sync.RWMutex

`RWMutex` позволяет:

- Несколько читателей одновременно (`RLock`)
- Только одного писателя за раз (`Lock`)

```
var mu sync.RWMutex
var balance int

func Balance() int {
    mu.RLock()           // блокировка для чтения
    defer mu.RUnlock()
    return balance
}

func Deposit(amount int) {
    mu.Lock()            // исключительная блокировка
    defer mu.Unlock()
    balance += amount
}
```

Правила

- `RLock()` / `RUnlock()` – для читателей.
- `Lock()` / `Unlock()` – для писателей.

Читатели не блокируют друг друга. Писатель блокирует всех (и читателей, и других писателей).

Когда использовать

`RWMutex` имеет смысл, если:

1. Чтений гораздо больше, чем записей.
2. Блокировка – узкое место (горутины часто ждут).

`RWMutex` медленнее обычного `sync.Mutex` из-за более сложной логики. Не используйте его без необходимости.

Важно

`RLock()` можно использовать только когда критический раздел точно не пишет в общие данные. Если сомневаешься – используйте обычный `Lock()`.

Ключевая мысль

`RWMutex` – это оптимизация для сценариев с большим количеством чтений и редкими записями. Для остальных случаев используйте обычный `sync.Mutex`.

Memory Synchronization

Проблема

В современных компьютерах несколько процессоров имеют собственные кеши. Записи в память могут буферизироваться и сбрасываться в разном порядке.

Без синхронизации горутина может читать устаревшие значения, даже если другая горутина уже записала новые.

Пример: недетерминированное поведение

```
var x, y int

go func() {
    x = 1
    fmt.Print("y:", y)
}()

go func() {
    y = 1
    fmt.Print("x:", x)
}()
```

Возможные выводы:

- y:0 x:1
- x:0 y:1
- x:1 y:1
- x:0 y:0

Почему так происходит

В отсутствие синхронизации:

- Компилятор может переставлять инструкции.
- Процессор может переставлять записи в память.
- Кеши разных ядер могут не синхронизироваться.

Решение

Синхронизация гарантирует порядок памяти.

Примитивы синхронизации (каналы, мьютексы) заставляют процессор сбросить все накопленные записи в основную память.

Правила

1. В пределах одной горутины порядок гарантирован.
2. Между горутинами – только через синхронизацию.

Ключевая мысль

Синхронизация нужна не только для порядка выполнения, но и для видимости изменений в памяти. Без неё программа может вести себя недетерминированно.

sync.Once

Проблема

Нужно выполнить инициализацию только один раз, но функция может быть вызвана параллельно из многих горутин.

Простая проверка `if icons == nil` не работает из-за гонок данных и переупорядочивания памяти.

Решение: sync.Once

```
var loadIconsOnce sync.Once
var icons map[string]image.Image

func Icon(name string) image.Image {
    loadIconsOnce.Do(loadIcons)
    return icons[name]
}
```

Как работает Do

- Первый вызов `Do(f)` выполняет `f()`.
- Последующие вызовы `Do(f)` ничего не делают.
- `Do` гарантирует, что `f()` выполнится только один раз, даже при параллельных вызовах.
- `Do` блокирует вызовы до завершения `f()`.

Почему лучше, чем мьютекс

Подход	Проблема
<code>sync.Mutex</code>	Блокирует всех даже после инициализации
<code>sync.RWMutex</code>	Сложный код, нужно перепроверять <code>nil</code>
<code>sync.Once</code>	Простой, безопасный, эффективный

Итог

`sync.Once` – идеальный инструмент для ленивой инициализации. Гарантирует, что функция выполнится ровно один раз, даже при параллельных вызовах, и все горутин увидят результат.

Race Detector

Concurrent Cache

Goroutines vs Threads